Bankers algorithm

```
n = int(input("Processes: "))
m = int(input("Resources: "))
a = [list(map(int, input().split())) for _ in range(n)]
mx = [list(map(int, input().split())) for _ in range(n)]
v = list(map(int, input("Available: ").split()))
need = [[mx[i][j]-a[i][j] for j in range(m)] for i in range(n)]
def safe():
    w, done, s = v[:], [0]*n, []
    while True:
        found = False
        for i in range(n):
            if not done[i] and all(need[i][j] <= w[j] for j in range(m)):
                w = [w[j]+a[i][j] for j in range(m)]
                done[i] = 1
                s.append("P"+str(i))
                found = True
        if not found:
            break
    return s if len(s) == n else None
while True:
    print("1.Need 2.Safe Sequence 3.Request 4.Exit")
    ch = int(input("Choice: "))
    if ch == 1:
        for row in need:
            print(row)
    elif ch == 2:
        print("Safe:", safe() or "Unsafe")
    elif ch == 3:
        p = int(input("Process (0 to n-1): "))
        r = list(map(int, input("Request: ").split()))
        if all(r[j] <= need[p][j] and r[j] <= v[j] for j in range(m)):
            v = [v[j]-r[j] for j in range(m)]
            a[p] = [a[p][j]+r[j] for j in range(m)]
            need[p] = [need[p][j]-r[j] for j in range(m)]
            if safe():
                print("Request Granted")
            else:
                v = [v[j]+r[j] for j in range(m)]
                a[p] = [a[p][j]-r[j] for j in range(m)]
                need[p] = [need[p][j]+r[j] for j in range(m)]
                print("Request Denied: Unsafe")
        else:
            print("Request Cannot Be Granted")
    elif ch == 4:
        break
```

page replacement algorithm

```
while True:
    print("1.FIFO 2.LRU 3.Optimal 4.Exit")
    ch = int(input("Choice: "))
    if ch == 4:
        break
    pages = list(map(int, input("Reference string: ").split()))
    n = int(input("Frames: "))
    frames, faults = [], 0
    for i, p in enumerate(pages):
        if p not in frames:
            faults += 1
            if len(frames) == n:
                if ch == 1:
                    frames.pop(0)
                elif ch == 2:
                    frames.pop(0)
                else:
                    future = pages[i+1:]
                    j = max(range(n), key=lambda x:
                        future.index(frames[x]) if frames[x] in future else float('inf'))
                    frames.pop(j)
            frames.append(p)
        elif ch == 2:
            frames.remove(p)
            frames.append(p)
        print(p, ":", frames)
    print("Page faults:", faults)
```

memory allocation

```
while True:
    print("\n1.First-Fit 2.Best-Fit 3.Worst-Fit 4.Exit")
    ch = int(input("Choice: "))
    if ch == 4:
        break
    if ch not in [1, 2, 3]:
        print("Invalid choice")
        continue
    b = list(map(int, input("Blocks: ").split()))
    p = list(map(int, input("Processes: ").split()))
    for i, size in enumerate(p):
        choices = [j for j in range(len(b)) if b[j] >= size]
        if not choices:
            print(f"P{i+1}: Not allocated")
            continue
        if ch == 1:
            j = choices[0]
        elif ch == 2:
            j = min(choices, key=lambda x: b[x])
        else:
            j = max(choices, key=lambda x: b[x])
        print(f"P{i+1} -> B{j+1}")
        b[j] -= size
```

semaphore

```
import threading
s = threading.Semaphore(1)
empty, full = threading.Semaphore(1), threading.Semaphore(0)
mutex, wrt = threading.Semaphore(1), threading.Semaphore(1)
item, readers = [], 0
def semaphore():
    with s:
        print("Critical section")
def producer():
    empty.acquire()
    item.append(10)
    print("Produced:", item[-1])
    full.release()
def consumer():
    full.acquire()
    print("Consumed:", item.pop())
    empty.release()
def reader():
    global readers
    mutex.acquire()
    readers += 1
    if readers == 1:
        wrt.acquire()
    mutex.release()
    print("Reading data")
    mutex.acquire()
    readers -= 1
    if readers == 0:
        wrt.release()
    mutex.release()
def writer():
    with wrt:
        print("Writing data")
def dining():
    with threading.Lock():
        print("Philosopher eating")
while True:
    print("\n1.Semaphore 2.Producer-Consumer")
    print("3.Readers-Writers 4.Dining Philosophers 5.Exit")
    ch = int(input("Choice: "))
    if ch == 1:
        semaphore()
    elif ch == 2:
        producer()
        consumer()
    elif ch == 3:
        reader()
        writer()
    elif ch == 4:
        dining()
    elif ch == 5:
        break
```

linux

1.Directory operations

```
Pwd
mkdir test
cd test
cd ..
rmdir test
ls
ls -l
ls -a
```

2. File operations

```
touch file.txt
cat file.txt
cp file.txt copy.txt
mv copy.txt new.txt
rm new.txt
```

3. Process commands

```
ps
ps -ef
top
sleep 100 &
jobs
kill PID
```

4. User and system commands

```
whoami
who
uptime
df -h
free -m
```

5. Redirection and piping

```
ls > files.txt
date >> files.txt
cat files.txt
ls | wc -l
```

Shell Scripting

1. Total, average and grade

```
#!/bin/bash
total=0
for i in {1..5}
do
   read -p "Mark: " m
   total=$((total+m))
done
avg=$((total/5))
echo "Total: $total"
echo "Average: $avg"
if [ $avg -ge 90 ]; then g=O
elif [ $avg -ge 80 ]; then g="A+"
elif [ $avg -ge 70 ]; then g=A
elif [ $avg -ge 60 ]; then g="B+"
elif [ $avg -ge 50 ]; then g=B
else g=Fail
fi
echo "Grade: $g"
```

2. File details

```
#!/bin/bash
read -p "Filename: " f
if [ -f "$f" ]; then
   wc "$f"
   head -5 "$f"
else
   echo "File does not exist"
fi
```

3. Menu-driven script

```
#!/bin/bash
while true
do
   echo "1.List files 2.Current directory"
   echo "3.Date and time 4.User 5.Exit"
   read -p "Choice: " ch
   case $ch in
      1) ls ;;
      2) pwd ;;
      3) date ;;
      4) whoami ;;
      5) exit ;;
      *) echo "Invalid choice" ;;
   esac
done
```

4. Even/odd and multiplication table

```
#!/bin/bash
read -p "Enter number: " n
if (( n % 2 == 0 )); then
   echo "Even"
else
   echo "Odd"
fi
for i in {1..10}
do
   echo "$n x $i = $((n*i))"
done
```

5. Welcome message and system details

```
#!/bin/bash
echo "Welcome!"
echo "User: $(whoami)"
echo "Date: $(date)"
echo "Directory: $(pwd)"
echo "Uptime: $(uptime -p)"
```

Process Management – C Programs

1. Display parent and child PID/PPID

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
   pid_t p = fork();
   if (p == 0)
      printf("Child: PID=%d PPID=%d\n", getpid(), getppid());
   else if (p > 0) {
      printf("Parent: PID=%d PPID=%d\n", getpid(), getppid());
      wait(NULL);
   }
   return 0;
}
```

2. Parent and child execute different statements

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
   pid_t p = fork();
   if (p == 0)
      printf("I am the child\n");
   else if (p > 0) {
      printf("I am the parent\n");
      wait(NULL);
   }
   return 0;
}
```

3. Create an orphan process

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main() {
   pid_t p = fork();
   if (p == 0) {
      printf("Before: PID=%d PPID=%d\n", getpid(), getppid());
      sleep(3);
      printf("After: PID=%d PPID=%d\n", getpid(), getppid());
   } else if (p > 0) {
      exit(0);
   }
   return 0;
}
```

4. Create a zombie process

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
   if (fork() == 0)
      exit(0);
   else {
      printf("Parent PID: %d\n", getpid());
      sleep(20);
   }
   return 0;
}
```

5. Parent waits for child using wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
   pid_t p = fork();
   if (p == 0) {
      printf("Child running\n");
   } else if (p > 0) {
      pid_t c = wait(NULL);
      printf("Parent waited for child: %d\n", c);
   }
   return 0;
}
```